DPDP COMPLIANCE AUDIT

medusa

Digital Personal Data Protection Act 2023

37/100

Grade F — Critical

Scanned: 13 April 2026
Files indexed: 10419
Languages: React, TypeScript, Node.js, Jest, Vite, ESLint

CONFIDENTIAL — Internal Use Only

DPDP Compliance Scan Report

Quick Actions for Developers

Fix these first to improve your compliance score:

• **DEBUG_MODE_ENABLED** → packages/medusa/src/commands/develop.ts
• **INCOMPLETE_DELETION_COVERAGE** (codebase-wide)
• **GRIEVANCE_OFFICER_MISSING** (codebase-wide)
• **CROSS_BORDER_TRANSFER_RISK** → ...product-import/helpers/import-template.ts
• **AUDIT_TRAIL_PARTIAL** (codebase-wide)

This codebase has 12 critical compliance gap(s) that require immediate attention under the Digital Personal Data Protection Act 2023. An additional 8 significant gap(s) were identified that should be addressed within 30 days. The estimated total remediation effort is 11.0–42.4 engineering days.

The application collects customer email, phone number, first name, last name, and order details. The biggest DPDP risk is the potential for unauthorized access to customer PII stored in the database due to the lack of explicit encryption mechanisms visible in the provided code.

Total findings: 28
• Rule Engine: 24
• AI Deep Review: 4

By Severity:
■ HIGH: 12
■ MEDIUM: 8
■ LOW: 2
■ INFO: 3
■ PASS: 3

Developer Priority:
■ Fix Now: 12
■ Fix This Sprint: 1

Tech Stack: React, TypeScript, Node.js, Jest, Vite
Risk Profile: The application collects customer email, phone number, first name, last name, and order details. The biggest DPDP risk is the potential for unauthorized access to customer PII stored in the database due to the lack of explicit encryption mechanisms visible in the provided code.

Compliance score by section

DPDP Section	Weight	Score	Status
Section 6	22	0/22 (0%)	FAIL
Section 6(6)	8	8/8 (100%)	PASS
Section 7	4	4/4 (100%)	PASS
Section 8(1)	18	0/18 (0%)	FAIL
Section 8	10	0/10 (0%)	FAIL
Section 8(3)	8	4.0/8 (50%)	WARN
Section 8(4)	8	4.0/8 (50%)	WARN
Section 8(6)	8	5.6/8 (70%)	WARN

Section 9	6	6/6 (100%)	PASS
Section 11	6	6/6 (100%)	PASS
Section 16	4	2.0/4 (50%)	WARN
Section 5	2	0/2 (0%)	FAIL
Section 13 — Grievance Redressal	4	0/4 (0%)	FAIL

Estimated Total Remediation Effort: 11.0–42.4 engineering days

Scan Coverage

Files analyzed: 10419

Data Flow Analysis

The following PII flow signals were inferred via static analysis (import/call adjacency). These are heuristic code-path chains, not proven runtime traces. Use them to guide developer review (confirm actual request → handler → service → database paths in your app).

Flow 1: Application Logs

PII fields: password

`create-db.ts → log-message.ts`

Flow 2: Application Logs

PII fields: password

`create-db.ts → log-message.ts → logger.ts`

Flow 3: Application Logs

PII fields: password

`create.ts`

Flow 4: Application Logs

PII fields: address

`customer.ts → client.ts`

Rule	Section	Severity	File
DEBUG_MODE_ENABLED	Section 8(1) — Security	HIGH	packages/medusa/src/commands/develop.ts
CROSS_BORDER_TRANSFER_RISK	Section 16 — Cross-Border Transfer	MEDIUM	...product-import/helpers/import-template.ts
INCOMPLETE_DELETION_COVERAGE	Section 8 — Data Principal Rights	HIGH	N/A
GRIEVANCE_OFFICER_MISSING	Section 13 — Grievance Redressal	HIGH	N/A
CONSENT_MISSING	Section 6 — Consent	HIGH	...[id]/@transfer-ownership/page.tsx
CONSENT_MISSING	Section 6 — Consent	HIGH	packages/admin/dashboard/src/routes/invite/invite.tsx
PASSWORD_NOT_HASHED	Section 8(1) — Security	HIGH	MULTIPLE
HARDCODED_SECRET	Section 8(1) — Security	HIGH	MULTIPLE

PLAINTEXT_PII_IN_LOGS	Section 8(1) — Security	HIGH	packages/design-system/toolbox/src/commands/tokens/command.ts
PLAINTEXT_PII_IN_LOGS	Section 8(1) — Security	HIGH	packages/medusa/src/migration-scripts/create-super-admin-role.ts
AUDIT_TRAIL_PARTIAL	Section 8(4) — Audit Trail	MEDIUM	N/A
NO_DELETION_MECHANISM	Section 8 — Data Principal Rights	MEDIUM	integration-tests/modules/helpers/create-authenticated-customer.ts
RETENTION_PARTIAL	Section 8(3) — Retention	LOW	N/A
NO_ERROR_HANDLING_IN_AUTH	Section 8(6) — Breach	MEDIUM	integration-tests/modules/helpers/create-authenticated-customer.ts
THIRD_PARTY_PII_SHARING	Section 5 — Notice	MEDIUM	packages/core/core-flows/src/order/workflows/create-fulfillment.ts
THIRD_PARTY_PII_SHARING	Section 5 — Notice	MEDIUM	packages/core/core-flows/src/order/workflows/create-order.ts
RETENTION_MISSING	Section 8(3) — Retention	MEDIUM	MULTIPLE
DATA_PORTABILITY_MISSING	Section 11 — Data Portability	LOW	N/A
INTERNAL_ROUTE_PII_ACCESS	Section 7 — Legitimate Use	INFO	packages/design-system/icons/src/components/payphone.tsx
INTERNAL_ROUTE_PII_ACCESS	Section 7 — Legitimate Use	INFO	packages/design-system/icons/src/components/credit-card.tsx
INTERNAL_ROUTE_PII_ACCESS	Section 7 — Legitimate Use	INFO	www/utils/packages/utils/src/examples-utils.ts
CONSENT_WITHDRAWAL_PRESENT	Section 6(6) — Consent Withdrawal	PASS	...workflow-execution-timeline-section/workflow-execution-timeline-section.tsx
BREACH_INDICATORS_ADEQUATE	Section 8(6) — Breach	PASS	N/A

DATA_ACCESS_PRESENT	Section 11 — Right to Access	PASS	packages/medusa/src/api/store/customers/middlewares.ts
DEEP_REVIEW_VALIDATED	Article 17 - Right to Erasure	MEDIUM	...brand/migrations/Migration20250805184935.ts
DEEP_REVIEW_VALIDATED	Article 5(1)(a) - Lawfulness, fairness and transparency	HIGH	packages/core/js-sdk/src/auth/index.ts
DEEP_REVIEW_VALIDATED	Article 5(1)(a) - Lawfulness, fairness and transparency	HIGH	packages/core/js-sdk/src/auth/index.ts
DEEP_REVIEW_VALIDATED	Article 5(1)(a) - Lawfulness, fairness and transparency	HIGH	packages/core/js-sdk/src/auth/index.ts

30-Day Remediation Roadmap

The following plan prioritises findings by severity and estimated effort. Complete Week 1 items before your next security review.

Week 1 — Critical (25–97 hours)

Fix immediately — these represent the highest legal exposure.

- Debug Mode Enabled — commands/develop.ts (4–16 hrs)
- Incomplete Deletion Coverage (4–16 hrs)
- Grievance Officer Missing (1–4 hrs)
- Consent Missing — @transfer-ownership/page.tsx (2–8 hrs)
- Consent Missing — invite/invite.tsx (2–8 hrs)
- Password Not Hashed — MULTIPLE (2–6 hrs)
- Hardcoded Secret — MULTIPLE (2–9 hrs)
- Plaintext Pii In Logs — tokens/command.ts (1–3 hrs)
- Plaintext Pii In Logs — migration-scripts/create-super-admin-role.ts (1–3 hrs)
- Deep Review Validated — auth/index.ts x3 (6–24 hrs)

Week 2 — High Priority (15–49 hours)

Address within 2 weeks — short fixes with significant compliance impact.

- Cross Border Transfer Risk — helpers/import-template.ts (4–12 hrs)
- Audit Trail Partial (2–8 hrs)
- No Deletion Mechanism — helpers/create-authenticated-customer.ts (2–6 hrs)
- No Error Handling In Auth — helpers/create-authenticated-customer.ts (1–3 hrs)
- Third Party Pii Sharing — workflows/create-fulfillment.ts (2–6 hrs)
- Third Party Pii Sharing — workflows/create-order.ts (2–6 hrs)
- Deep Review Validated — migrations/Migration20250805184935.ts (2–8 hrs)

Week 3 — Significant (45–181 hours)

Schedule in sprint — requires more engineering time.

- Retention Missing — MULTIPLE (45–181 hrs)

Week 4 — Advisory (3–12 hours)

Address before next compliance review.

- ■ Retention Partial (1–4 hrs)
- ■ Data Portability Missing (2–8 hrs)

Total estimated remediation effort: 88–339 hours (11.0–42.4 engineering days)

Changes Since 2026-04-08

▼ Score: 56.0 → 37 (-19.0 points)

✓ 12 finding(s) resolved
■ 12 new finding(s) introduced
— 12 finding(s) unchanged
■ 2278 file(s) changed

Resolved since last scan:

- ✓ Third Party Pii Sharing
- ✓ Retention Missing
- ✓ Retention Missing
- ✓ Internal Route Pii Access
- ✓ Internal Route Pii Access
- ✓ Internal Route Pii Access
- ✓ Deep Review Validated
- ✓ Deep Review Validated

New findings this scan:

- ■ Grievance Officer Missing [HIGH]
- ■ Consent Missing [HIGH]
- ■ Consent Missing [HIGH]
- ■ No Deletion Mechanism [MEDIUM]
- ■ Third Party Pii Sharing [MEDIUM]
- ■ Third Party Pii Sharing [MEDIUM]
- ■ Retention Missing [MEDIUM]
- ■ Data Portability Missing [LOW]

Details

Finding 1 of 28 — ■ FIX NOW

[Section 8(1) — Security]

1. [HIGH] DEBUG_MODE_ENABLED — packages / medusa / src / commands / develop.ts:82

Evidence snippet: NODE_ENV: "development",

Estimated fix time: 4–16 hours

Debug mode is enabled. In production, this exposes interactive debuggers (Flask/Werkzeug), detailed stack traces, and internal system information to users. Flask debug mode specifically allows arbitrary code execution via the Werkzeug console. DPDP Section 8(1) requires appropriate security measures to protect personal data.

Risk:

Enabling debug mode in a production environment can expose sensitive system information and potentially allow arbitrary code execution, violating DPDP Section 8(1) security requirements and leading to data breaches.

Fix:

1. Ensure the `develop` command is never used in a production environment.
2. Implement environment variable checks to disable debug features when `NODE_ENV` is 'production'.
3. Review the `develop` command's implementation for any explicit debug flags and ensure they are conditionally disabled in production.
4. Add automated tests to verify that debug mode is disabled in production builds.

DPDP:

Section 8(1) — Security

Example:

```
import { MedusaCommand } from "@medusajs/utils" new MedusaCommand({ // ... other configurations // Ensure debug mode is disabled if NODE_ENV is 'production' // Example: // debug: process.env.NODE_ENV !== 'production' }).run()
```

Finding 2 of 28 — ■ REVIEW & FIX

[Section 16 — Cross-Border Transfer]

2. [MEDIUM] CROSS_BORDER_TRANSFER_RISK — packages / admin / dashboard / src / routes / products / product-import / helpers / import-template.ts:4

Evidence snippet: ,coffee-mug-v3,Medusa Coffee Mug,,Every programmer's best friend.,published,https://medusa-public-images.s3.eu-west-1.amazonaws.com/coffee-mug.png,400,,,,,,,,,,,TRUE,,,One Size,,,FALSE,TRUE,,,,,,,,

Estimated fix time: 4–12 hours

Foreign cloud storage/CDN detected with PII models in repo. Cross-border transfer risk.

Risk:

The application may be transferring PII to foreign cloud storage or CDNs without adequate safeguards, which is a violation of DPDP Section 16 requirements for cross-border data transfers.

Fix:

1. Identify all third-party services used for file storage or content delivery.
2. Review the data processing agreements with these third-party providers to ensure they meet DPDP cross-border transfer requirements.
3. If PII is processed, implement data anonymization or pseudonymization before transfer, or ensure SCCs/BCRs are in place.
4. Document the legal basis and safeguards for all cross-border data transfers.

DPDP:

Section 16 — Cross-Border Transfer

Example:

```
import axios from 'axios'; const uploadFileToCloud = async (file: File) => { const formData = new FormData(); formData.append('file', file); try { // Replace with your secure cloud storage endpoint const response = await axios.post('/api/upload-to-secure-cloud', formData, { headers: { 'Content-Type': 'multipart/form-data', }, }); return response.data; } catch (error) { console.error('File upload failed:', error); throw error; } }
```

Finding 3 of 28 — ■ FIX NOW

[Section 8 — Data Principal Rights]

3. [HIGH] INCOMPLETE_DELETION_COVERAGE — N / A

Estimated fix time: 4–16 hours

Deletion mechanism detected but may not cover all PII storage. Coverage: 24% of inferred PII entities (86/359 including FK dependents). Uncovered: account_transaction, admin_views_entity_column, admin_views_entity_configuration, admin_views_entity_configurations_active, admin_views_entity_configurations_id. DPDP Section 8 requires complete erasure across all systems.
Entity deletion coverage: 24% (86 covered / 273 uncovered)
Uncovered entities: account_transaction, admin_views_entity_column, admin_views_entity_configuration, admin_views_entity_configurations_active, admin_views_entity_configurations_id, base_repository_find, campaign_budget, campaign_budget_usage, and 265 more
Entity→file hints: account_transaction: models/account-transaction.ts; admin_views_entity_column: paths/admin_views_{entity}_columns.yaml; admin_views_entity_configuration: paths/admin_views_{entity}_configurations.yaml; admin_views_entity_configurations_active: paths/admin_views_{entity}_configurations_active.yaml; admin_views_entity_configurations_id: paths/admin_views_{entity}_configurations_{id}.yaml
Risk:
The absence of a comprehensive deletion mechanism means that personal data may persist in the system longer than necessary, violating DPDP Section 8 requirements for data minimization and erasure.

Fix:
1. Identify all data entities that store or process personal data, including related foreign key dependencies.

2. Implement a robust deletion service that handles the complete removal of data across all relevant tables and associated records.

3. Develop a mechanism to track and verify the successful deletion of data for all PII entities.

4. Integrate this deletion service into user-facing features (e.g., account deletion) and backend processes (e.g., data retention policies).

DPDP:
Section 8 — Data Principal Rights

Example:
```
import { EntityManager } from '@medusajs/typeorm'; export const deleteCustomerData = async (manager: EntityManager, customerId: string): Promise<void> => { // Example: Delete customer orders first await manager.delete('order', { customer_id: customerId }); // Example: Delete customer addresses await manager.delete('address', { customer_id: customerId }); // Example: Delete the customer itself await manager.delete('customer', { id: customerId }); // Add deletions for all other related PII entities }
```

Finding 4 of 28 — ■ FIX NOW

[Section 13 — Grievance Redressal]

[NEW]

4. [HIGH] GRIEVANCE_OFFICER_MISSING — N / A

Estimated fix time: 1–4 hours

No compliant Grievance Officer / DPO publication found. A PASS requires grievance-related text or a dedicated route **and** a contact email or dedicated grievance/DPO URL. Tiny repos without a grievance route are not marked PASS.

Risk:

The lack of a designated Grievance Officer or DPO contact point prevents data subjects from exercising their rights to seek redress for data protection concerns, as required by DPDP Section 13.

Fix:

1. Designate a specific individual or role as the Grievance Officer or Data Protection Officer (DPO).

2. Create a dedicated page or section on the website that clearly publishes the contact information (email or URL) for the Grievance Officer/DPO.

3. Ensure this contact information is easily accessible to all users.

4. Update the privacy policy to reference the designated Grievance Officer/DPO and their contact details.

DPDP:

Section 13 — Grievance Redressal

Example:

```
// Example: Add a new route for DPO contact // In your routing configuration (e.g., Express.js or FastAPI) app.get('/privacy/dpo', (req, res) => { res.json({ message: 'Contact our Data Protection Officer', email: 'dpo@example.com', // or a URL to a contact form/page // url: '/contact-dpo' }); }); // Update privacy policy text to include: // 'You can contact our Data Protection Officer at dpo@example.com or visit /privacy/dpo for more information.'
```

Finding 5 of 28 — ■ FIX NOW

[Section 6 — Consent]

[NEW]

5. [HIGH] CONSENT_MISSING — packages / plugins / draft-order / src / admin / routes / draft-orders / [id] / @transfer-ownership / page.tsx:70

Evidence snippet: ? `${name} (${order.customer.email})`

Estimated fix time: 2–8 hours

Personal data collection and API endpoint detected with no consent mechanism.

Risk:

The application displays customer email addresses without a clear consent mechanism, potentially violating the DPDP Act's requirement for explicit consent before processing personal data.

Fix:

1. Implement a consent banner or checkbox before displaying customer PII on this admin page, clearly stating what data is shown and why.

2. Update the `useDraftOrder` hook or the data fetching layer to include a consent status check before retrieving and displaying customer details.

3. If this is an internal administrative view, document the legal basis for processing under Section 7 (legitimate use) and ensure access is strictly controlled.

DPDP:

Section 6(1) — Consent requires explicit, informed, and freely given consent for the collection and processing of personal data.

Example:

```
// Assuming a consent context is available const { hasConsented } = useConsent(); if (!hasConsented) { return <p>Consent is required to view customer details.</p>; } // ... rest of the component rendering customer details
```

Finding 6 of 28 — ■ FIX NOW

[Section 6 — Consent]

[NEW]

6. [HIGH] CONSENT_MISSING — packages / admin / dashboard / src / routes / invite / invite.tsx:13

Evidence snippet: import { useSignUpWithEmailPass } from "../../hooks/api/auth"

Estimated fix time: 2–8 hours

Personal data collection and API endpoint detected with no consent mechanism.

Risk:

The application collects email and password information during the invite/sign-up process without an explicit consent mechanism, potentially violating the DPDP Act's requirements for data processing.

Fix:

1. Add a mandatory checkbox labeled 'I agree to the terms and privacy policy' before the user can submit the invite/sign-up form.

2. Ensure the terms and privacy policy clearly outline the collection and use of email and password data.

3. If this is an internal administrative view, document the legal basis for processing under Section 7 (legitimate use) and ensure access is strictly controlled.

DPDP:

Section 6(1) — Consent requires explicit, informed, and freely given consent for the collection and processing of personal data.

Example:

```
import React from 'react'; import { Form } from '../../components/common/form'; import * as z from 'zod'; import { useForm } from 'react-hook-form'; import { zodResolver } from '@hookform/resolvers/zod'; const InviteSchema = z.object({ email: z.string().email(), // ... other fields consent: z.literal(true, { errorMap: () => ({ message: 'Consent is required' }) }), }); // Inside your component: const form = useForm<z.infer<typeof InviteSchema>>({ resolver: zodResolver(InviteSchema), defaultValues: { consent: false }, }); // Add a checkbox input for consent in your JSX <Form.Field control={form.control} name="consent" render={({ field }) => ( <Form.Checkbox checked={field.value} onCheckedChange={field.onChange}> I agree to the terms and privacy policy </Form.Checkbox> )} />
```

Finding 7 of 28 — ■ FIX NOW

[Section 8(1) — Security]

7. [HIGH] PASSWORD_NOT_HASHED — MULTIPLE

Estimated fix time: 2–6 hours

Affected files (4):

• packages/cli/create-medusa-app/src/utils/create-db.ts
• packages/core/utils/src/common/handle-postgres-database-error.ts
• packages/core/utils/src/modules-sdk/load-module-database-config.ts
• packages/medusa/src/commands/db/create.ts

Model file has password field but no hash/bcrypt/argon/pbkdf2/scrypt/encrypt reference. Detected in 4 files: create-db.ts, handle-postgres-database-error.ts, load-module-database-config.ts, create.ts

Risk:

The codebase lacks explicit references to password hashing mechanisms (like bcrypt, Argon2, etc.) for user authentication, creating a significant security risk if passwords are stored or transmitted insecurely.

Fix:

1. Implement password hashing using a strong algorithm (e.g., bcrypt, Argon2) in the user registration and password update API endpoints before storing passwords in the database.

2. Ensure that all authentication-related mutations (e.g., `useSignUpWithEmailPass`) in `packages/admin/dashboard/src/hooks/api/auth.tsx` are handled by backend services that perform hashing.

3. Review the database schema and any data access layers to confirm that plain-text passwords are not being persisted.

4. Add a dependency on a password hashing library (e.g., `bcrypt`) to your backend project if not already present.

DPDP:

Section 8(1) — Security requires implementing appropriate technical and organizational measures to protect personal data against unauthorized access, disclosure, alteration, or destruction.

Example:

```
// Example for a backend API endpoint (e.g., in Express.js) const bcrypt = require('bcrypt'); const saltRounds = 10; app.post('/register', async (req, res) => { const { email, password } = req.body; const hashedPassword = await bcrypt.hash(password, saltRounds); // Store hashedPassword in the database // ... });
```

Finding 8 of 28 — ■ FIX NOW

[Section 8(1) — Security]

8. [HIGH] HARDCODED_SECRET — MULTIPLE:37

Evidence snippet: * password: "supersecret"

Estimated fix time: 2–9 hours

Affected files (13):

• packages/core/js-sdk/src/admin/invite.ts
• packages/core/js-sdk/src/auth/index.ts
• www/utils/generated/oas-output/operations/admin/post_admin_invites_accept.ts
• www/utils/generated/oas-output/operations/admin/post_auth_[actor_type]_[auth_provider].ts
• www/utils/generated/oas-output/operations/admin/post_auth_[actor_type]_[auth_provider]_register.ts
• www/utils/generated/oas-output/operations/admin/post_auth_[actor_type]_[auth_provider]_update.ts
• ... and 7 more files

Hardcoded hardcoded_password detected. Detected in 13 files: invite.ts, index.ts, post_admin_invites_accept.ts, post_auth_[actor_type]_[auth_provider].ts and 9 more

Risk:

Hardcoded secrets, such as the 'secret' passphrase used for JWT key generation, pose a significant security risk, potentially allowing unauthorized access to sensitive data and system functions if compromised.

Fix:

1. Remove the hardcoded 'secret' passphrase from `integration-tests/http/__fixtures__/auth/medusa-config.js`.

2. Store the passphrase securely in environment variables (e.g., `process.env.JWT_PASSPHRASE`).

3. Update the `generateKeyPairSync` and module configurations to use the passphrase from the environment variable.

4. Ensure that all sensitive configuration values are managed via environment variables or a dedicated secrets management system.

DPDP:

Section 8(1) — Security requires implementing appropriate technical and organizational measures to protect personal data against unauthorized access, disclosure, alteration, or destruction.

Example:

```
// In medusa-config.js // const passphrase = "secret"; // Remove this line const passphrase = process.env.JWT_PASSPHRASE; // ... rest of the configuration using passphrase
```

Finding 9 of 28 — ■ FIX NOW

[Section 8(1) — Security]

9. [HIGH] PLAINTEXT_PII_IN_LOGS — packages / design-system / toolbox / src / commands / tokens / command.ts:181

Evidence snippet: logger.info(`Writing typography tokens to file`)

Estimated fix time: 1–3 hours

PII keyword appears in logging call. Personal data may be written to logs in plaintext.

Risk:

Personal data may be logged in plaintext, violating the requirement for data security and potentially exposing sensitive information if logs are compromised or accessed inappropriately.

Fix:

1. Review all logging statements in `packages/medusa/src/migration-scripts/create-super-admin-role.ts` and ensure no PII is passed as arguments to the logger.

2. Implement a PII detection mechanism or a strict logging policy to prevent PII from being logged.

3. Consider using a logging library that supports PII masking or redaction before writing to logs.

DPDP:

Section 8(1) — Security

Example:

```
import { logger } from "@medusajs/utils" // ... other code ... const sensitiveData =
"user@example.com"; // Example PII // Before: // logger.info(`Processing user:
${sensitiveData}`); // After (if sensitiveData is PII and should not be logged): //
logger.info(`Processing user: [REDACTED]`); // Or, if it's a known field that needs masking:
// logger.info(`Processing user with ID: ${userId}`); // Assuming userId is not PII
```

Finding 10 of 28 — ■ FIX NOW

[Section 8(1) — Security]

10. [HIGH] PLAINTEXT_PII_IN_LOGS — packages / medusa / src / migration-scripts / create-super-admin-role.ts:49

Evidence snippet: logger.info(` ${index + 1}. ${user.email || user.id} (${user.id})`)

Estimated fix time: 1–3 hours

PII keyword appears in logging call. Personal data may be written to logs in plaintext.

Risk:

Personal data may be logged in plaintext, violating the requirement for data security and potentially exposing sensitive information if logs are compromised or accessed inappropriately.

Fix:

1. Review all logging statements in `packages/medusa/src/migration-scripts/create-super-admin-role.ts` and ensure no PII is passed as arguments to the logger.

2. Implement a PII detection mechanism or a strict logging policy to prevent PII from being logged.

3. Consider using a logging library that supports PII masking or redaction before writing to logs.

DPDP:

Section 8(1) — Security

Example:

```
import { logger } from "@medusajs/utils" // ... other code ... const sensitiveData =
"user@example.com"; // Example PII // Before: // logger.info(`Processing user:
${sensitiveData}`); // After (if sensitiveData is PII and should not be logged): //
logger.info(`Processing user: [REDACTED]`); // Or, if it's a known field that needs masking:
// logger.info(`Processing user with ID: ${userId}`); // Assuming userId is not PII
```

Finding 11 of 28 — ■ REVIEW & FIX

[Section 8(4) — Audit Trail]

11. [MEDIUM] AUDIT_TRAIL_PARTIAL — N / A

Estimated fix time: 2–8 hours

Partial audit trail detected. Found: change tracking. Missing: access logging, audit model.

Risk:

The absence of comprehensive audit trails, specifically missing access logging and a dedicated audit model, means there's no clear record of who accessed what data and when, hindering accountability and the ability to investigate potential breaches.

Fix:

1. Implement a robust access logging mechanism that records user authentication events, data access attempts (read, write, delete), and the timestamp of these actions.

2. Define and implement an audit model (e.g., an 'AuditLog' entity) to store these access events, including user ID, action performed, target resource, and timestamp.

3. Integrate the audit logging into relevant parts of the application, such as API endpoints, data modification functions, and user authentication flows.

4. Ensure audit logs are stored securely and retained according to policy, with mechanisms to prevent tampering.

DPDP:

Section 8(4) — Audit Trail

Example:

```
import { logger } from "@medusajs/utils"; // Assume an AuditLog model and repository exist // import { AuditLog } from "../models/audit-log"; // import { AuditLogRepository } from "../repositories/audit-log"; async function logAccess(userId: string, action: string, resource: string) { try { // In a real app, you'd use a repository to save this to the DB logger.info(`Audit: User ${userId} performed ${action} on ${resource}`); // await AuditLogRepository.create({ userId, action, resource, timestamp: new Date() }); } catch (error) { logger.error(`Failed to log audit event: ${error.message}`); } } // Example usage in an API endpoint: // app.get('/users/:id', async (req, res) => { // const userId = req.user.id; // const targetUserId = req.params.id; // await logAccess(userId, 'READ', `User profile ${targetUserId}`); // // ... rest of the logic // });
```

Finding 12 of 28 — ■ REVIEW & FIX

[Section 8 — Data Principal Rights]

[NEW]

12. [MEDIUM] NO_DELETION_MECHANISM — integration-tests / modules / helpers / create-authenticated-customer.ts

Estimated fix time: 2–6 hours

Deletion logic exists but only in internal/non–user-facing routes. Data principals need a way to request erasure (e.g. account settings, DELETE /me or /account).

Risk:

While a deletion mechanism exists within the authentication service, it's used in an integration test helper and not exposed as a user-facing feature, meaning data principals lack a direct way to request erasure of their personal data.

Fix:

1. Expose a user-facing endpoint (e.g., `DELETE /me` or `DELETE /account`) that allows authenticated users to request the deletion of their account and associated personal data.

2. Implement the logic within the `AuthService` or a dedicated customer service to handle these deletion requests, ensuring all associated PII is removed or anonymized.

3. Document the data deletion process and communicate it to users through privacy policies or account management interfaces.

4. Consider adding a confirmation step (e.g., email verification) before permanent deletion to prevent accidental data loss.

DPDP:

Section 8 — Data Principal Rights

Example:

```
import { Router } from "express"; const router = Router(); // Assume authService and customerService are injected or available router.delete('/me', async (req, res) => { const userId = req.user.id; // Assuming user ID is available from authentication middleware try { await authService.delete(userId); // Or a more specific customer deletion service await customerService.delete(userId); res.status(204).send(); // No Content } catch (error) { res.status(500).json({ message: 'Failed to delete account' }); } }); export default router;
```

Finding 13 of 28 — ■ BACKLOG

[Section 8(3) — Retention]

13. [LOW] RETENTION_PARTIAL — N / A

Estimated fix time: 1–4 hours

Retention signals found in 166 model(s) but 73 PII model(s) have no retention policy. Uncovered: address.ts, AdminCreateFulfillment.yaml, AdminCustomerAddress.yaml.

Risk:

The absence of data retention policies for PII, such as addresses, means that personal data may be stored indefinitely, increasing the risk of unauthorized access or misuse over time and violating DPDP Section 8(3). This can lead to significant privacy breaches and regulatory penalties.

Fix:

1. Implement a data retention policy framework within the application. This could involve adding a 'retention_period' field to relevant PII models (e.g., address models) and creating a background job to periodically review and delete data exceeding its retention period.

2. Define specific retention periods for different types of PII, such as addresses, in a central configuration file or database table.

3. Integrate a mechanism to automatically purge or anonymize data that has passed its defined retention period, ensuring compliance with DPDP Section 8(3).

4. Document the data retention policies and procedures in the application's privacy policy and internal documentation.

DPDP:

Section 8(3) — Personal data shall not be kept for longer than is necessary for the purposes for which the personal data are processed.

Example:

```
typescript // Example: Adding retention period to an address model (conceptual) interface Address { id: string; street_1: string; city: string; postal_code: string; country_code: string; // ... other address fields retention_period_days?: number; // New field for retention policy } // Example: Background job to purge old addresses (conceptual) async function purgeOldAddresses() { const cutoffDate = new Date(); // Assume 'retention_period_days' is defined for each address or a default is used cutoffDate.setDate(cutoffDate.getDate() - DEFAULT_ADDRESS_RETENTION_DAYS); // Logic to find and delete addresses older than cutoffDate // await db.addresses.deleteMany({ where: { createdAt: { lt: cutoffDate } } }); }
```

Finding 14 of 28 — ■ REVIEW & FIX

[Section 8(6) — Breach]

14. [MEDIUM] NO_ERROR_HANDLING_IN_AUTH — integration-tests / modules / helpers / create-authenticated-customer.ts:11

Evidence snippet: The `api.post` call to `REDACTED_POSSIBLE_SECRET` is not wrapped in a try-catch block, meaning any errors during this authentication step would not be explicitly handled or logged.

Estimated fix time: 1–3 hours

Auth routes have no try/catch or try/except blocks. Errors may go unlogged.

Risk:

Authentication routes lack explicit error handling (try-catch blocks), which can lead to unhandled exceptions and potential security vulnerabilities if errors are not logged or managed properly. This failure to handle errors can result in system instability and make it difficult to diagnose and respond to security incidents as required by DPDP Section 8(6).

Fix:

1. Wrap the `api.post` call on line 11 in a try-catch block to handle potential errors during the initial signup process.

2. Wrap the `api.post` call on line 21 in a try-catch block to handle potential errors during customer creation.

3. Wrap the `api.post` call on line 30 in a try-catch block to handle potential errors during the customer sign-in process.

4. Inside each catch block, log the error details using a suitable logging mechanism (e.g., `console.error` or a dedicated logger) and re-throw a custom error or return an appropriate error response.

DPDP:

Section 8(6) — A data controller shall implement appropriate technical and organizational measures to ensure the security of personal data, including protection against unauthorized or unlawful processing and against accidental loss, destruction or damage.

Example:

```
typescript try { const signup = await api.post(REDACTED_POSSIBLE_SECRET, { email, password = "REDACTED_CREDENTIAL", }); } catch (error) { console.error("Signup failed:", error); // Handle or re-throw the error as appropriate throw new Error("Authentication failed during signup."); }
```

Finding 15 of 28 — ■ REVIEW & FIX

[Section 5 — Notice]

[NEW]

15. [MEDIUM] THIRD_PARTY_PII_SHARING — packages / core / core-flows / src / order / workflows / create-fulfillment.ts:245

Evidence snippet: const shippingAddress = order.shipping_address ?? { id: undefined }

Estimated fix time: 2–6 hours

Personal data present in file that imports third-party SDK. Data may be shared without user awareness or notice (DPDP Section 5).

Risk:

The application processes and potentially shares PII (shipping address) with an inventory adjustment module, which may be a third-party service. This sharing could occur without adequate user notice or consent, violating DPDP Section 5. Failure to provide notice can erode user trust and lead to non-compliance.

Fix:

1. Review the `adjustInventoryLevelsStep` and its underlying implementation to determine if it involves any third-party services that process PII.

2. If `adjustInventoryLevelsStep` or any other part of the fulfillment process involves sharing PII with third parties, ensure that a clear notice is provided to the user about this data sharing, as required by DPDP Section 5.

3. Implement mechanisms to obtain explicit user consent before sharing PII with third-party services, if consent is the legal basis for processing.

4. Consider anonymizing or pseudonymizing PII before sending it to third-party services, if feasible and appropriate for the service's functionality.

DPDP:

Section 5 — A data controller shall give notice to the data subject about the collection, processing, and sharing of personal data.

Example:

```
typescript // Assuming 'adjustInventoryLevelsStep' might interact with a third-party service // and 'order.shipping_address' is considered PII. // Before passing shippingAddress to a potentially external service: const shippingAddress = order.shipping_address ?? { id: undefined }; // delete shippingAddress.id; // Consider if ID is truly needed by the external service // If shippingAddress is sent to a third-party service: // Ensure notice is given and consent is obtained if required. // Example: // if (thirdPartyServiceRequiresAddress(shippingAddress)) { // await notifyUserAboutDataSharing(order.customer_id, 'shipping_address'); // await getConsentForDataSharing(order.customer_id, 'shipping_address'); // } // delivery_address: shippingAddress as any, // Pass processed address
```

Finding 16 of 28 — ■ REVIEW & FIX

[Section 5 — Notice]

[NEW]

16. [MEDIUM] THIRD_PARTY_PII_SHARING — packages / core / core-flows / src / order / workflows / create-order.ts:62

Evidence snippet: const shippingAddress = data.input.shipping_address ?? { id: undefined }

Estimated fix time: 2–6 hours

Personal data present in file that imports third-party SDK. Data may be shared without user awareness or notice (DPDP Section 5).

Risk:

The application processes and potentially shares PII (email, shipping and billing addresses) with internal workflows or external services for order creation and adjustments. This processing might occur without adequate user notice or consent, violating DPDP Section 5. Lack of transparency can lead to non-compliance and damage user trust.

Fix:

1. Review the `refreshDraftOrderAdjustmentsWorkflow` and any other workflows or services involved in order creation to identify if PII is being shared with third parties.

2. If PII is shared with third parties, ensure that users are clearly notified about this data sharing as per DPDP Section 5.

3. Implement mechanisms to obtain explicit user consent before sharing PII with third-party services, if consent is the chosen legal basis for processing.

4. Consider anonymizing or pseudonymizing PII before it is processed by internal workflows or external services, if feasible and appropriate.

DPDP:

Section 5 — A data controller shall give notice to the data subject about the collection, processing, and sharing of personal data.

Example:

```
typescript // Example: Handling PII (email, addresses) during order creation. // Ensure that if 'data.input.shipping_address', 'data.input.billing_address', or 'data.input.email' // are sent to any third-party services or external APIs, proper notice and consent mechanisms are in place. // Example for email: // if (externalServiceRequiresEmail(data_.email)) { // await notifyUserAboutDataSharing(data_.customer_id, 'email'); // await getConsentForDataSharing(data_.customer_id, 'email'); // } // Similar checks and actions for shippingAddress and billingAddress. // const shippingAddress = data.input.shipping_address ?? { id: undefined }; // delete shippingAddress.id; // const billingAddress = data.input.billing_address ?? { id: undefined }; // data_.email = data.input?.email ?? data.customerData.customer.email;
```

Finding 17 of 28 — ■ REVIEW & FIX

[Section 8(3) — Retention]

[NEW]

17. [MEDIUM] RETENTION_MISSING — MULTIPLE

Estimated fix time: 45–181 hours

Affected files (73):

• packages/plugins/draft-order/src/admin/lib/schemas/address.ts
• www/apps/api-reference/specs/admin/components/schemas/AdminCreateFulfillment.yaml
• www/apps/api-reference/specs/admin/components/schemas/AdminCustomerAddress.yaml
• www/apps/api-reference/specs/admin/components/schemas/AdminCustomerAddressRespons
e.yaml
• www/apps/api-reference/specs/admin/components/schemas/AdminDraftOrder.yaml
• www/apps/api-reference/specs/admin/components/schemas/AdminDraftOrderPreview.yaml
• ... and 67 more files

Model/schema file contains PII fields but no retention or expiry signals detected. Detected in 73 files: address.ts, AdminCreateFulfillment.yaml, AdminCustomerAddress.yaml, AdminCustomerAddressResponse.yaml and 69 more

Risk:

The application collects and processes personal data, including names and contact information, but lacks explicit mechanisms for data retention or expiry. This can lead to the indefinite storage of personal data, increasing the risk of unauthorized access and violating DPDP Section 8(3) obligations.

Fix:

1. Implement a data retention policy and mechanism within the application. This could involve adding fields to schemas (e.g., `created_at`, `updated_at`, `expires_at`) and creating background jobs or API endpoints to periodically clean up or anonymize data that has exceeded its retention period.

2. For each PII field or data entity, define a clear retention period based on business and legal requirements.

3. Document the data retention policy and procedures in the application's privacy policy and internal documentation.

4. Consider implementing data anonymization or pseudonymization techniques for data that needs to be retained for longer periods but does not require direct personal identification.

DPDP:

Section 8(3) — Retention: Data Fiduciaries shall not retain personal data for longer than is necessary for the purpose for which it is processed.

Example:

```
import { z } from "zod" export const addressSchema = z.object({ country_code:
z.string().min(1), first_name: z.string().min(1), last_name: z.string().min(1), address_1:
z.string().min(1), address_2: z.string().nullish(), company: z.string().nullish(), city:
z.string().min(1), province: z.string().nullish(), postal_code: z.string().min(1), phone:
z.string().nullish(), // Add retention fields createdAt: z.date().default(new Date()),
expiresAt: z.date().optional(), })
```

Finding 18 of 28 — ■ BACKLOG

[Section 11 — Data Portability]

[NEW]

18. [LOW] DATA_PORTABILITY_MISSING — N / A

Estimated fix time: 2–8 hours

No data export or portability endpoint detected. DPDP Section 11 requires Data Fiduciaries to provide personal data in a commonly used, machine-readable format upon request.

Risk:

The application does not appear to offer a mechanism for users to export or receive their personal data in a machine-readable format. This absence violates DPDP Section 11, which mandates data portability upon request, potentially leading to user dissatisfaction and non-compliance penalties.

Fix:

1. Implement an API endpoint (e.g., `/users/me/export`) that allows authenticated users to request an export of their personal data.

2. This endpoint should query the database for all PII associated with the authenticated user and format it into a commonly used, machine-readable format such as JSON or CSV.

3. Provide a mechanism for the user to download the exported data, either directly via a file download or through a secure link.

4. Ensure the export process is efficient and handles potentially large amounts of data gracefully.

DPDP:

Section 11 — Data Portability: Data Fiduciaries shall provide the personal data of the Data Principal in a structured, commonly used and machine-readable format.

Example:

```
import express from 'express'; import fs from 'fs'; import path from 'path'; const router = express.Router(); router.get('/export-data', async (req, res) => { const userId = req.user.id; // Assuming user is authenticated and ID is available const userData = await fetchUserData(userId); // Replace with your data fetching logic const jsonData = JSON.stringify(userData, null, 2); const filePath = path.join(__dirname, `user-data-${userId}.json`); fs.writeFileSync(filePath, jsonData); res.download(filePath, err => { if (err) { console.error(err); res.status(500).send('Error exporting data'); } fs.unlinkSync(filePath); // Clean up the file after download }); }); export default router;
```

Finding 19 of 28 — ■ BACKLOG

[Section 7 — Legitimate Use]

[NEW]

19. [INFO] INTERNAL_ROUTE_PII_ACCESS — packages / design-system / icons / src / components / payphone.tsx

Internal file (payphone.tsx) accesses PII. Verify access is restricted to authorised contexts and PII is not exposed to logs or error responses.

Finding 20 of 28 — ■ BACKLOG

[Section 7 — Legitimate Use]

[NEW]

20. [INFO] INTERNAL_ROUTE_PII_ACCESS — packages / design-system / icons / src / components / credit-card.tsx

Internal file (credit-card.tsx) accesses PII. Verify access is restricted to authorised contexts and PII is not exposed to logs or error responses.

Finding 21 of 28 — ■ BACKLOG

[Section 7 — Legitimate Use]

[NEW]

21. [INFO] INTERNAL_ROUTE_PII_ACCESS — www / utils / packages / utils / src / examples-utils.ts

Internal service/utility accesses PII. Verify: (1) this file is only called from authorised contexts, (2) PII is not inadvertently exposed to logs or error messages.

Finding 22 of 28 — ■ BACKLOG

[Section 6(6) — Consent Withdrawal]

22. [PASS] CONSENT_WITHDRAWAL_PRESENT — packages / admin / dashboard / src / routes / workflow-executions / workflow-execution-detail / components / workflow-execution-timeline-section / workflow-execution-timeline-section.tsx

Consent withdrawal mechanism detected. Found in: packages/admin/dashboard/src/routes/workflow-executions/workflow-execution-detail/components/workflow-execution-timeline-section/workflow-execution-timeline-section.tsx (line 105)

Finding 23 of 28 — ■ BACKLOG

[Section 8(6) — Breach]

23. [PASS] BREACH_INDICATORS_ADEQUATE — N / A

Logging and/or breach alerting present.

Finding 24 of 28 — ■ BACKLOG

[Section 11 — Right to Access]

[NEW]

24. [PASS] DATA_ACCESS_PRESENT — packages / medusa / src / api / store / customers / middlewares.ts

Data access endpoint detected for authenticated users.

Finding 25 of 28 — ■ FIX THIS SPRINT

[Article 17 - Right to Erasure]

25. [MEDIUM] DEEP_REVIEW_VALIDATED [AI Deep Review] — integration-tests / modules / src / modules / brand / migrations / Migration20250805184935.ts

Estimated fix time: 2–8 hours

The 'brand' table migration includes a 'deleted_at' column and an index on it for soft deletes. However, the 'down' method only drops the table, which is a hard delete and bypasses the soft delete mechanism.

Finding 26 of 28 — ■ FIX NOW

[Article 5(1)(a) - Lawfulness, fairness and transparency]

26. [HIGH] DEEP_REVIEW_VALIDATED [AI Deep Review] — packages / core / js-sdk / src / auth / index.ts

Estimated fix time: 2–8 hours

The example payload for the `register` method includes a plaintext password: `password = "REDACTED_CREDENTIAL"`.

Finding 27 of 28 — ■ FIX NOW

[Article 5(1)(a) - Lawfulness, fairness and transparency]

27. [HIGH] DEEP_REVIEW_VALIDATED [AI Deep Review] — packages / core / js-sdk / src / auth / index.ts

Estimated fix time: 2–8 hours

The example payload for the `login` method includes a plaintext password: `password = "REDACTED_CREDENTIAL"`.

Finding 28 of 28 — ■ FIX NOW

[Article 5(1)(a) - Lawfulness, fairness and transparency]

28. [HIGH] DEEP_REVIEW_VALIDATED [AI Deep Review] — packages / core / js-sdk / src / auth / index.ts

Estimated fix time: 2–8 hours

The example payload for the `callback` method includes a plaintext password: `password = "REDACTED_CREDENTIAL"`.

AI Deep Compliance Review (Delta — Changed Files Only)

This section shows AI analysis that identified compliance gaps beyond deterministic rules. Validated findings are integrated into the main findings list above with [AI Deep Review] labels.

Overall Risk: HIGH

The application's handling of customer PII presents a significant risk due to the potential for unauthorized access and exposure. Inconsistent data deletion practices and the inclusion of sensitive information in analytics events and example payloads undermine our data protection posture. Addressing these vulnerabilities is critical to maintaining user trust and DPDP compliance.

Estimated remediation: 15 engineering day(s)

> **Most Critical Action:** Remove all plaintext password examples from JSDoc comments in the Authentication & Session layer and ensure all sensitive data is properly masked or omitted in logging and analytics events.

• **PII Exposure in Analytics and Logging** [HIGH] — Article 6 (Principles relating to processing of personal data) — layers: Third-Party & PII Flows, API Routes & Controllers
• **Inconsistent Data Handling and Deletion** [MEDIUM] — Article 17 (Right to erasure) — layers: Data Models & Storage, Third-Party & PII Flows

1. [HIGH] Plaintext password in example payload — Authentication & Session — Article 5 (Principles relating to processing of personal data)
Example payloads for register, login, and callback methods include plaintext passwords, directly contradicting secure handling of credentials.

2. [HIGH] Unnecessary PII in PostHog event properties — API Routes & Controllers — Article 6 (Principles relating to processing of personal data)
PostHog events capture `$current_url` and `$raw_user_agent`, which can potentially contain sensitive user information.

3. [MEDIUM] PII potentially exposed in markdown content — Third-Party & PII Flows — Article 6 (Principles relating to processing of personal data)
User agent and URL information are captured via PostHog when the 'accept' header includes 'text/plain' or 'text/markdown', potentially exposing sensitive data.

4. [MEDIUM] Inconsistent deletion handling for 'brand' table — Data Models & Storage — Article 17 (Right to erasure)
The 'brand' table migration supports soft deletes, but the 'down' method performs a hard delete, bypassing the intended data erasure mechanism.

5. [MEDIUM] Potential for sensitive data in query params — Authentication & Session — Article 5 (Principles relating to processing of personal data)
The 'callback' method accepts a generic 'query' parameter which could inadvertently include sensitive information passed directly to the API route.

Layer Breakdown
Third-Party & PII Flows — 3 finding(s). The primary compliance concern in this layer relates to the potential exposure of sensitive information through analytics tracking in markdown content rendering. Additionally, a data minimization concern exists within the draft order update workflow.
Data Models & Storage — 2 finding(s). The reviewed migrations show an inconsistency in deletion handling for the 'brand' table, potentially impacting data erasure compliance. Feature flags are used to control migration execution, which warrants careful management to avoid compliance risks.
API Routes & Controllers — 4 finding(s). The reviewed files primarily handle serving markdown content. The main compliance concern identified is the potential for PII leakage through analytics events sent to PostHog, specifically the capture of full URLs and user agent strings. No other significant DPDP control gaps were identified in this delta review.
Authentication & Session — 4 finding(s). The Authentication & Session layer primarily handles token management and authentication flows. While core authentication mechanisms appear to be in place, the examples provided in the JSDoc comments demonstrate a recurring issue with displaying plaintext credentials, which poses a risk to data protection principles.